

Persistence (II)

File System - 2

中国人民大学信息学院

卞昊穹

bianhq@ruc.edu.cn

内部资料，请勿外传

Outline

- Linux I/O Stack
- File System
- File System Implementation
- **Fast File System**
- FSCK and Journaling

Fast File System

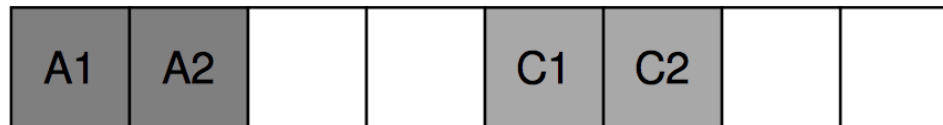
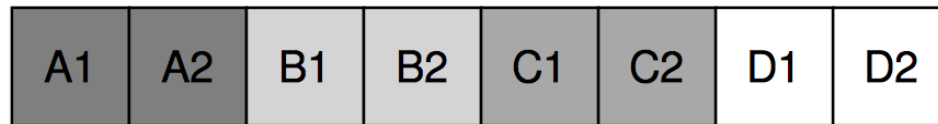
- Old Unix File System
 - Ken Thompson (UNIX wizard)
 - Super block - inode - data



- Poor performance
 - 吞吐量相当于磁盘raw i/o的2%！！！！
 - 因为把磁盘当做了随机访问的内存，没有针对性的优化

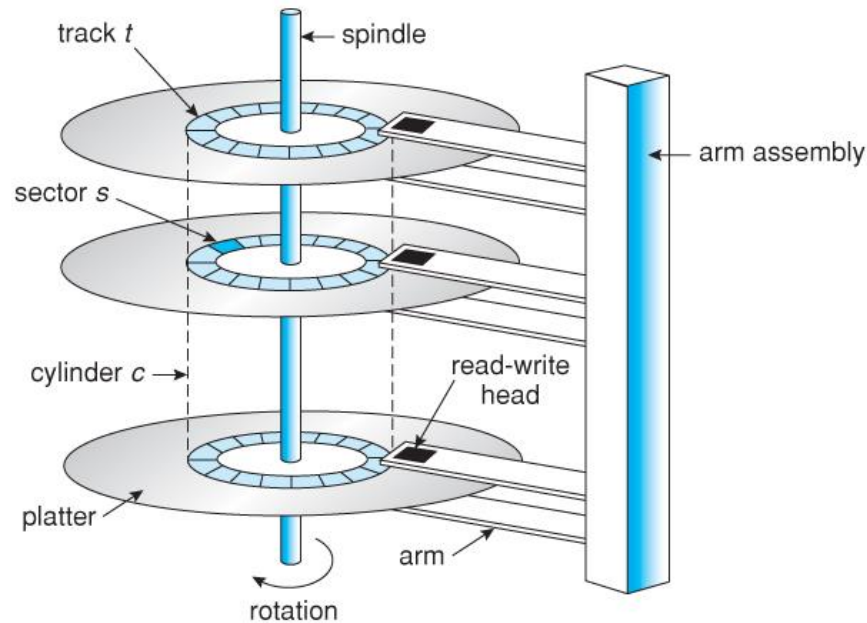
Fast File System

- Old Unix File System
 - 性能不好的另一个原因: quite fragmented
 - Free space管理过于简单
 - 同一个文件的数据分散, 磁头来回移动
 - 例如, **ABCD**四个文件, **BD**删除后, 写入新文件**E**



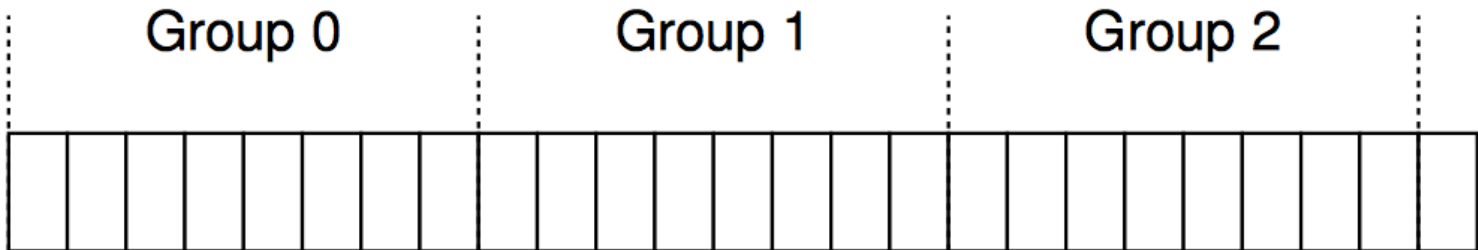
Fast File System (FFS)

- FFS: Disk-aware
 - **Cylinder Group:** FFS基于CHS addressing, 将连续（相邻）的N个cylinder看做一个cylinder group



Fast File System

- FFS: Disk-aware
 - Cylinder group也可以看做是一组连续的Block/sector
 - 例如，8个block组成一个group（下图）
 - 访问同一个group中的block比较快，因为磁头移动很少



Data Locality of Reference

- 数据访问的局部性(locality)假设:
 - 数据中的某些部分之间存在某种联系，这些相关联的部分会经常在一段时间内被一起访问。例如：
 - FS中经常按照super block->bitmaps->inode->data blocks的顺序访问块设备
 - 同一个目录下的多个文件经常在较短的时间内被访问
 - 同一个文件中“距离”较近的data blocks经常被连续访问

如何利用Cylinder Group来优化性能？

Fast File System

- FFS: Disk-aware
 - cylinder group:
 - 每个group都包括super block, 可以并发访问
 - 也包括组内的inode bitmap, data bitmap, 和inode table
 - 文件的元数据和数据放在临近的位置, 访问会比较快



Fast File System

- 核心问题：如何放置文件到cylinder group?
 - *"keep related stuff together, keep unrelated stuff far apart."*
 - 但FS中数据是动态变化的
 - 对于目录
 - 将新目录放在一个目录比较少、free inodes比较多的cg中
 - 对于文件
 - 同一个文件的inode和data blocks放在同一个group
 - 同目录下的文件与目录放在同一个group

System优化的两个要点：

- 1) 适应硬件特点（尤其是重点考虑缺点）；
- 2) 适应上层应用的特征（workload）

Fast File System

- 同目录下的文件与目录放在同一个group
 - 例如：假设每个group包括10个inode，10个data block
 - 目录有3个：/, /a, /b
 - 文件有4个：/a/c, /a/d, /a/e, /b/f
 - FFS放置方案

group	inodes	data
0	/-----	/-----
1	acde-----	accddee---
2	bf-----	bff-----
3	-----	-----
4	-----	-----
5	-----	-----
6	-----	-----
7	-----	-----
...		

Fast File System

- 对比：简单的放置策略

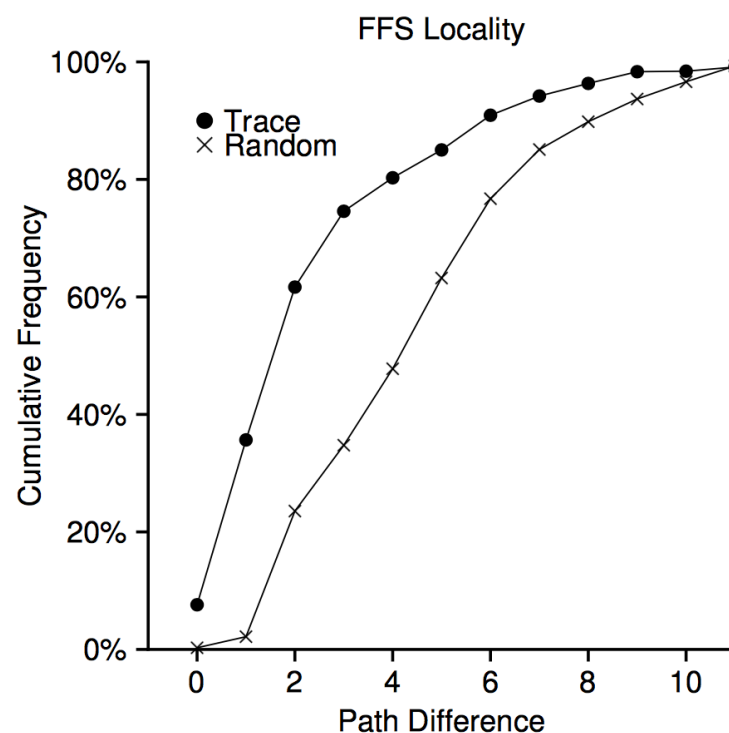
group	inodes	data
0	/-----	/-----
1	a-----	a-----
2	b-----	b-----
3	c-----	cc-----
4	d-----	dd-----
5	e-----	ee-----
6	f-----	ff-----
7	-----	-----
...		

两种策略下，访问同一个目录中的文件，分别需要访问多少个**group**？

Fast File System

- **Measuring File Locality**

- 连续两次打开文件的**目录深度**统计
 - 例如先后打开 `/dir/f`, `/dir/g`, 距离 (**path difference**) 为1
- 结论：多数连续文件访问的目录深度差距不大，局部性好
 - **40%**是同一个文件或同一个目录的文件
 - **25%**是两级目录以内
 - 所以，**FFS**的局部性假设成立



Fast File System

- The **Large** File Exception

- 不使用large file exception

- /a占用了大部分data blocks, /下没有空间用来创新新文件了

```
group inodes      data
  0 /a----- /aaaaaaaaa aaaaaaaaaa aaaaaaaaaa a-----
  1 -----
  2 -----
  ...
```

- 解决办法: 把大文件的data blocks分散到多个不同的cylinder group

```
group inodes      data
  0 /a----- /aaaaa-----
  1 ----- aaaaa-----
  2 ----- aaaaa-----
  3 ----- aaaaa-----
  4 ----- aaaaa-----
  5 ----- aaaaa-----
  6 -----
  ...
```

Fast File System

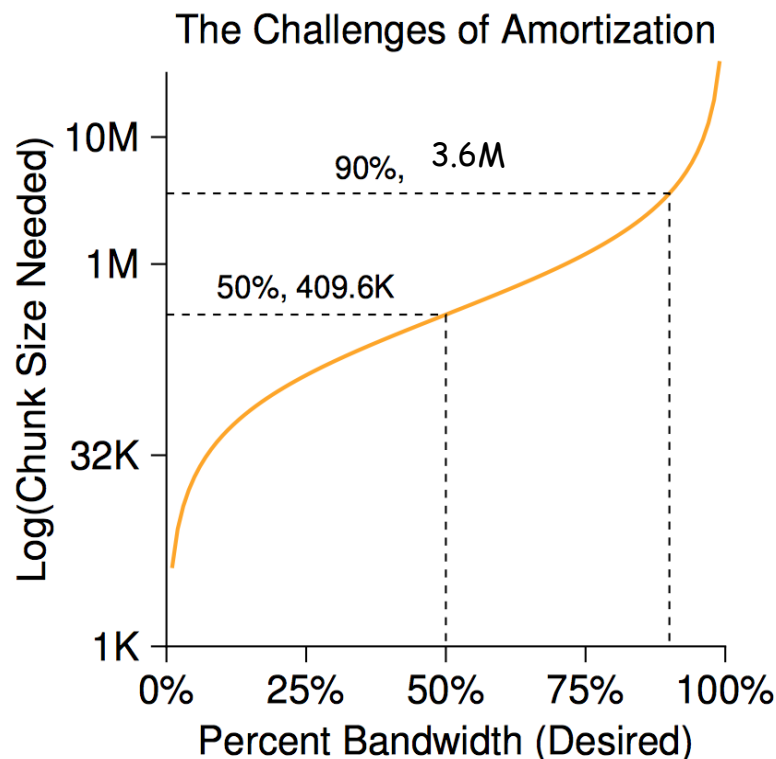
- The Large File Exception

- 问题：数据分散，性能下降
- 解决：合理设置chunk大小，获得“足够好”的性能
 - 50%带宽：409.6KB
 - 90%带宽：3.6MB
 - 99%带宽：39.6MB

计算过程：

设disk sequential read bandwidth=40MB/s，即40.96KB/ms，disk positioning time=10ms，则：
 $\text{target_ratio} = (10\text{ms} * 40.96\text{KB/ms}) / \text{chunk_size}$

target_ratio=1/9时，chunk_size=3.6MB



代价分摊 (Cost Amortization) !

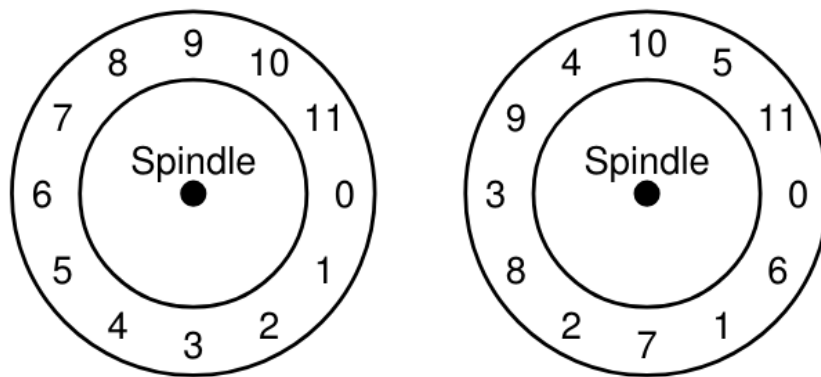
Another Example of Cost Amortization

- HDFS (Hadoop Distributed File System):
 - 每个大文件（如**100GB**）拆分成多个数据块（**block**）
 - **blocks**分散存储在不同的机器上
 - **client**找到并打开一个**block**需要一定的时间
 - 如何选择**block**大小，保证顺序读取一个大文件的效率？

Fast File System

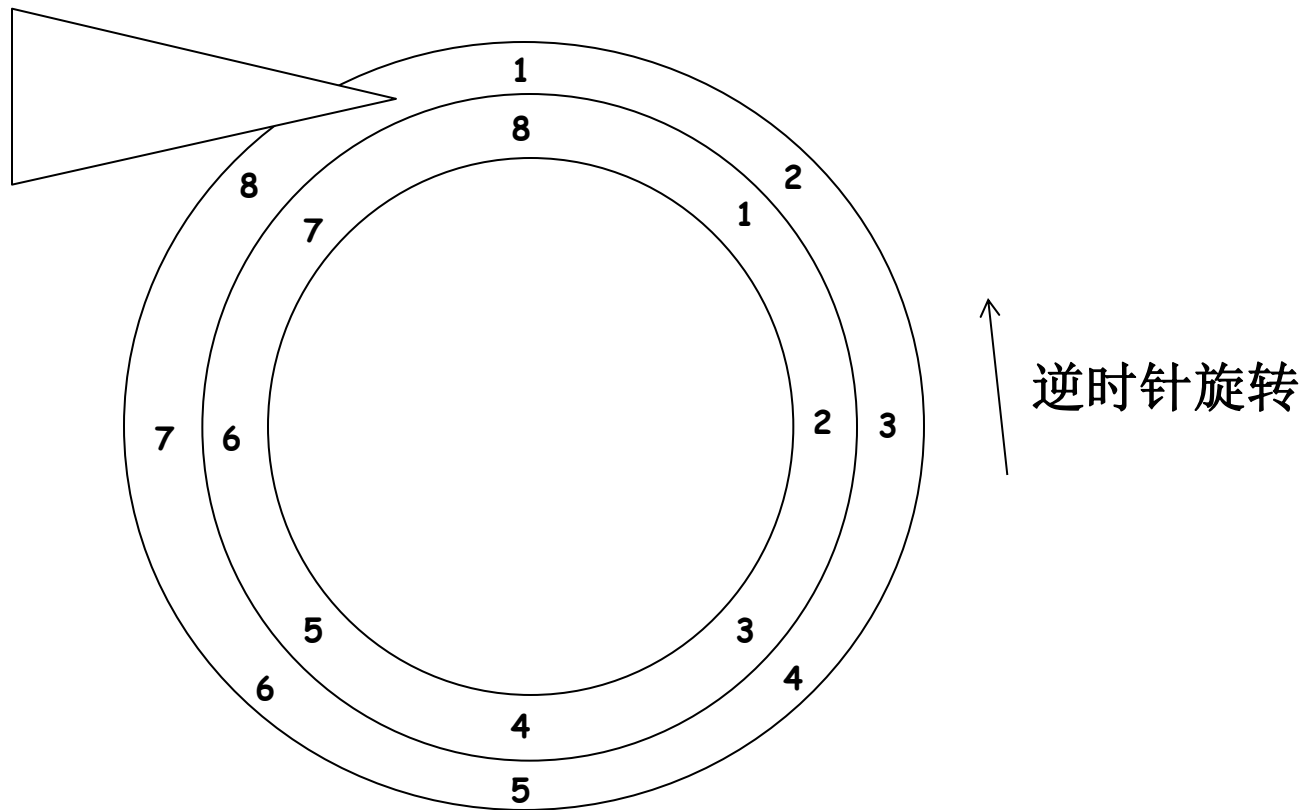
- 小文件的问题 (<2KB)
 - **internal fragmentation**
 - 无法充分利用4KB的data block, 空间浪费
 - 解决: **sub-block (512B)**
 - 先以sub-block为单位分配空间
 - 文件空间如果增长超过2KB, 则迁移到一个完整的block上
 - 新的问题: 额外的数据拷贝
 - 解决: 修改libc, 缓冲写数据, 再分配到4KB的块上

Reading the next block



- FFS: 间隔放置data blocks, 这会浪费磁盘带宽
- 更好的解决方案: read-ahead
 - 可以由disk, block I/O layer, 或者FS完成

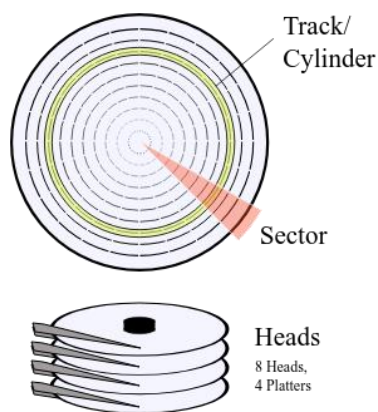
Reading the next cylinder



磁臂(**arm**)从外圈**cylinder**向内圈**cylinder**移动, 至少需要个**settle time**, 如**2ms**, 为了使**arm**移动到内圈**cylinder**时磁头(**head**)刚好读到第一个**sector**, 内圈第一个**sector**需要错后一些

Seek time in a FS backed by HDD

- 磁盘（HDD）在FS看来是一个一维Block向量
- FS中的seek实际是从block向量中的一个position调到另一个position
- FS中的seek time如何计算？



如果两个data block在同一个，只有spin time（对于7200rpm磁盘，0-8.33ms）

如果两个data block相距2-4个cylinder， $\text{seek time} = \text{spin time} + \text{settle time}$

如果两个data block相距稍远一点？

如果两个data block相距很远？

总体上，seek time与seek distance正相关

How to place data in a file?

- Use a file system that supports extent (e.g., Ext4, XFS)
 - Extent ensures a large byte range in the file is store sequentially in the disk
- Put frequently co-accessed data chunks closer in a file

Outline

- Linux I/O Stack
- File System
- File System Implementation
- Fast File System
- **FSCK and Journaling**
- Log-structured File Systems
- Data Integrity and Protection

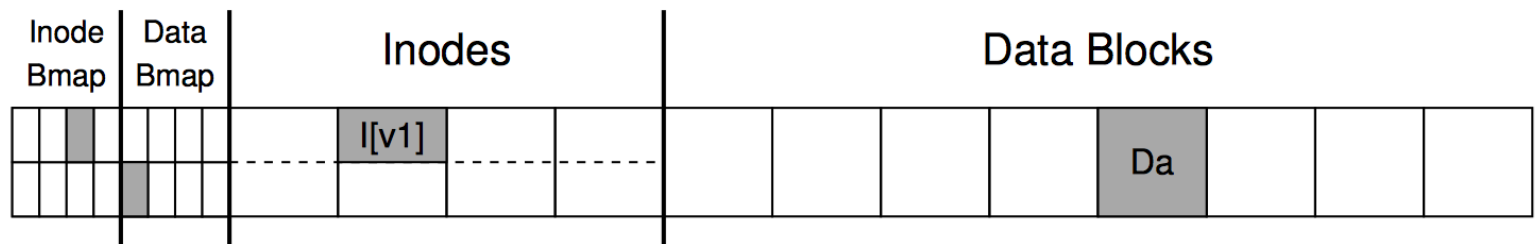
FSCK and Journaling

- 可容忍的系统状态：
 - 完全正常、完全失效（e.g., RAM）
 - 部分失效【存储/数据库、分布式系统】
- **Crash-consistency problem**
 - 例如连续2次写入高度关联的A和B，在中间发生crash
 - 原子性问题
 - 系统进入一个inconsistent状态
 - 解决方法
 - FSCK: file system checker
 - Journaling: write-ahead log

FSCK and Journaling

- 一个例子
 - 一个文件大小为1，只包括一个数据块
 - **inode**部分域:

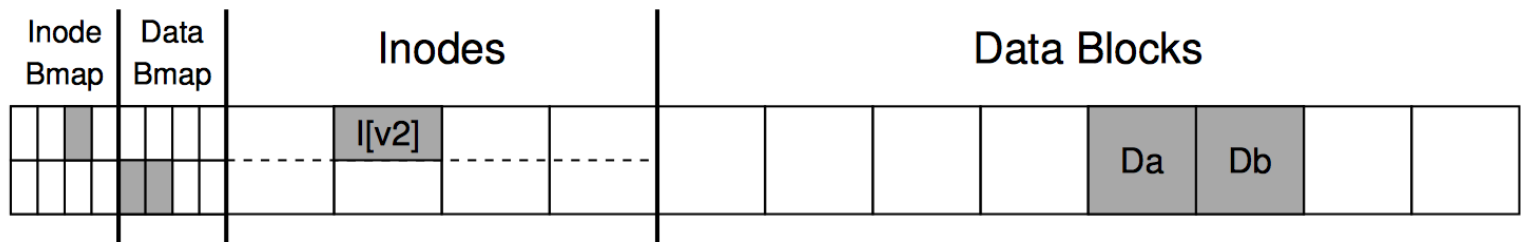
owner	: remzi
permissions	: read-write
size	: 1
pointer	: 4
pointer	: null
pointer	: null
pointer	: null



FSCK and Journaling

- 一个例子
 - 后面文件又追加写入一个block Db
 - **inode**部分域:

owner	:	remzi
permissions	:	read-write
size	:	2
pointer	:	4
pointer	:	5
pointer	:	null
pointer	:	null



FSCK and Journaling

- 三个具体的写操作
 - Bitmap
 - Inode
 - Data block (Db)
- 先写入page cache，延迟写入磁盘
 - 在写盘之前计算机crash，有多种可能：

有几种inconsistent的可能？

FSCK and Journaling

- **#1. Just the data block (Db) is written to disk**
 - 文件的数据在磁盘上
 - 但是**inode**没有记录，**bitmap**也没有
 - 不是问题：
 - 相当于**Db**没有写入，不会引入不一致的问题

FSCK and Journaling

- **#2. Just the updated inode (I[v2]) is written to disk**
 - 问题1: 按照inode信息, 会从data block 5位置读出垃圾数据
 - 问题2: file-system inconsistency
 - Inode告诉我们data block 5有数据
 - Bitmap告诉我们data block 5没有数据

FSCK and Journaling

- #3. Just the updated bitmap (B[v2]) is written to disk
 - 还是bitmap和inode不一致
 - Space leak
 - Data block被占用，但是不会被利用

FSCK and Journaling

- #4. inode ($I[v2]$) and bitmap ($B[v2]$) written, but not data (Db)
 - 元数据一致，但是数据是垃圾数据

FSCK and Journaling

- #5. inode ($I[v2]$) and data block (Db) written, but not the bitmap ($B[v2]$)
 - 读取正确，但数据可能被覆盖

FSCK and Journaling

- #6. bitmap ($B[v2]$) and data block (Db) written, but not the inode ($I[v2]$)
 - 元数据不一致
 - 无法读取db

FSCK and Journaling

Crash不可避免

- 一致性问题本质
 - 完成一个文件写操作，需要多次**device**写操作
 - 这写**device**写操作应该具有原子性
 - 要么都成功，要么都不成功

FSCK and Journaling

- Solution #1: The File System Checker
 - Fck - UNIX tool
 - Scan the blocks, fix problems at best effort
 - 目标：让元数据及数据恢复一致
 - 在文件系统被**mount**之前检查
 - 避免**fsck**的同时，**fs**修改数据和元数据

FSCK and Journaling

- Solution #1: The File System Checker
 - Fscck检查什么？
 - **Superblock:**
 - 检查superblock看起来是不是正常
 - » 如果不正常，则通过备份恢复
 - » 恢复不了怎么办？

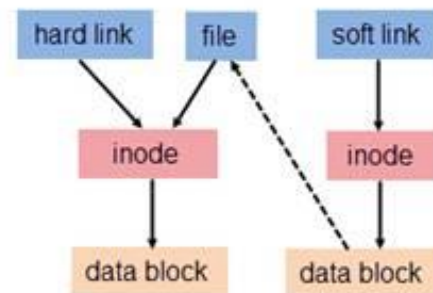
FSCK and Journaling

- Solution #1: The File System Checker
 - Fscck检查什么？
 - **Free blocks:**
 - 检查inode, indirect blocks, double indirect blocks, 看数据在磁盘上如何分布；
 - 然后基于这些信息去纠正bitmaps和inodes之间不一致的问题
 - » 如果不一致，则以inodes为准（凭啥？）

FSCK and Journaling

- Solution #1: The File System Checker
 - Fscck检查什么?
 - **Inode state:**
 - 检查每个inode看起来是否正常;
 - 如果错误纠正不了, 就会删除该inode, 同时更新inode bitmap

FSCK and Journaling



- Solution #1: The File System Checker
 - Fscck检查什么?
 - Inode links:
 - inode的link count属性记录了有多少个目录包含指向该文件的name-inode mapping
 - 扫描整个目录树，重新计算link count，纠正inode记录的link count;
 - 如果一个inode的link count是0，则将其放到lost+found目录下

FSCK and Journaling

- Solution #1: The File System Checker
 - Fscck检查什么？
 - **Deduplicated pointers:**
 - 检查是否存在多个不同的Inode指向同一个data/pinter block;
 - » 如果其中某个Inode本身不正确，则将其清除
 - » 对于剩余的inode，复制block

FSCK and Journaling

- Solution #1: The File System Checker
 - Fscck检查什么？
 - **Bad block pointers:**
 - 检查inode和pointer block中的block pointers是否有问题，例如指向一个越界的block
 - » 从inode或pointer block中清除这些bad pointers

FSCK and Journaling

- Solution #1: The File System Checker
 - Fscck检查什么？
 - **Directory Check:**
 - **directory**有自己的**data block**，其中的数据结构是由**FS**自己定义的，因此可以被**fsck**理解

FSCK and Journaling

- Solution #1: The File System Checker
 - FSCK的问题:
 - 扫描文件系统中的全部**blocks**，太慢了！
 - 有一些不一致问题无法修复

FSCK and Journaling

- Solution #2: Journaling (Write-Ahead Logging)
 - WAL来自于database领域
 - 最早使用journaling的文件系统是Cedar(1987)
 - 现在广范使用: Linux ext3, ext4, reiserfs, IBM's JFS, SGI's XFS, Windows NTFS

电影推荐

- Revolution OS (操作系统革命)
 - 2001年上映的纪录片电影
 - 记录了GNU、Linux、自由软件运动、开源软件运动的兴起
 - Richard Stallman、Linus Torvalds等真人主演



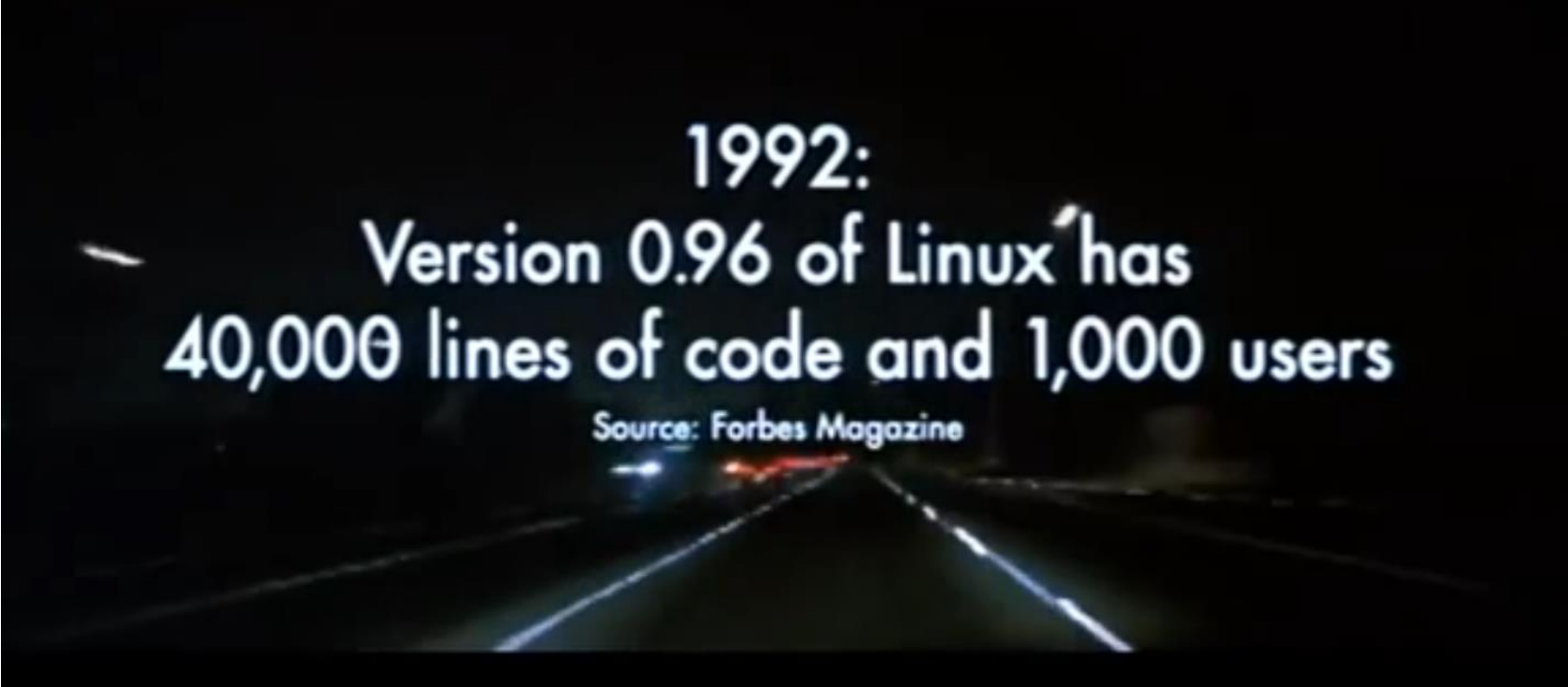
<https://www.bilibili.com/video/BV1gx411Q79H/>



The term for it is "monolithic",
Linux是单一内核的操作系统
the OS itself is one entity, indivisible.
也就是说操作系统本身是一个不可分的整体
uh, while in the microkernel,
它不像微内核系统那样,
the, the operating system kernel is actually
微内核系统实际上是一组服务,
uh, just a collection of servers that
它们做不同的工作,
do different things and then they have a common protocol
彼此通过一个共同的协议来通讯,
for doing communication between themselves.
来协调它们的动作.

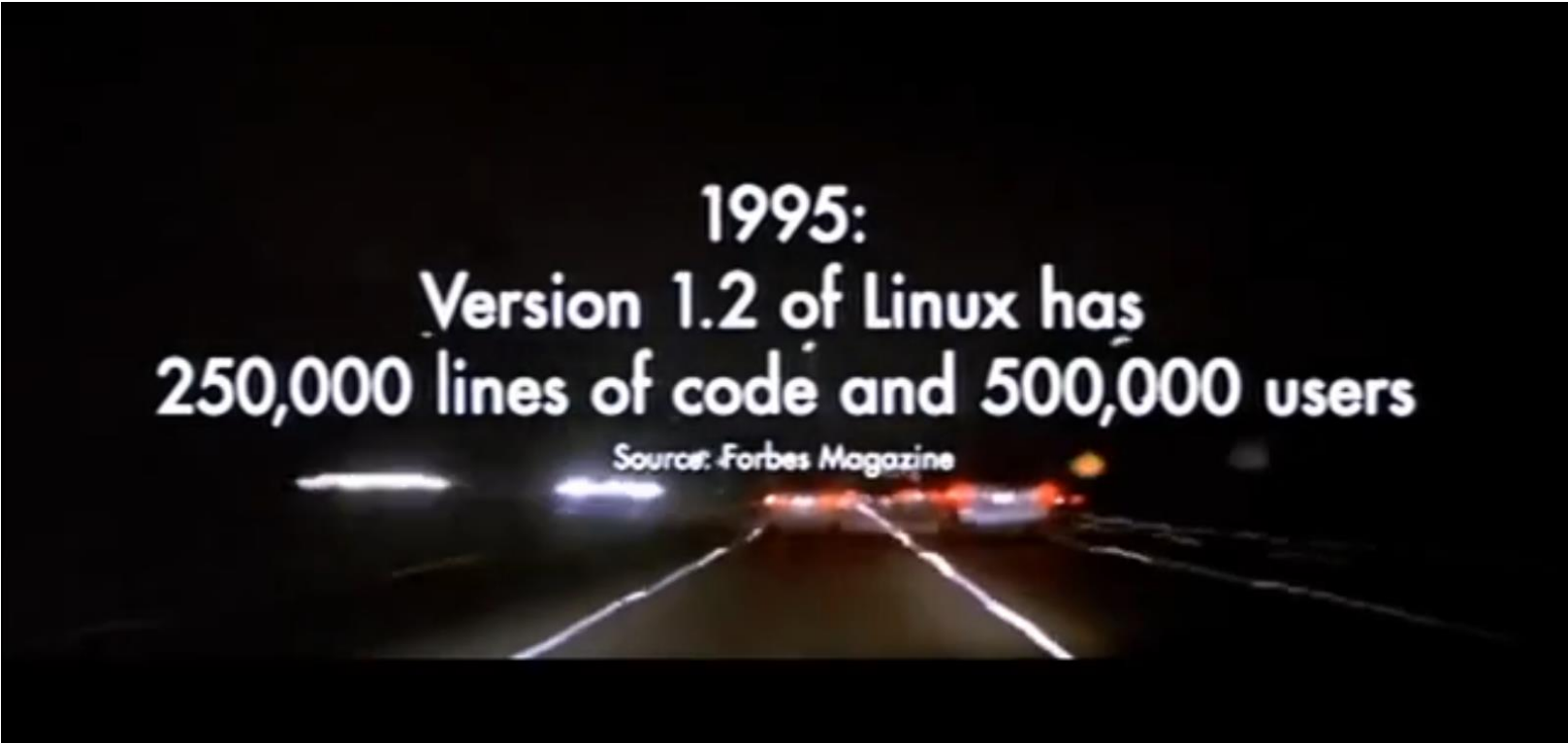
1992:
Version 0.96 of Linux has
40,000 lines of code and 1,000 users

Source: Forbes Magazine





The killer app of Linux was undoubtedly
the Apache web server.
Linux里最招人喜欢的软件无疑就是Apache



1995:
Version 1.2 of Linux has
250,000 lines of code and 500,000 users

Source: Forbes Magazine

中国**1994**年**5**月接入互联网，**1995**年**9**月第一个**ISP**



1995年MySQL发布， 1996年PostgreSQL发布



1998:
Version 2.110 of Linux has 1.5 million
lines of code and 7.5 million users

Source: Forbes Magazine

and new people porting, new users

??

1999年前后，中国最早一批国产数据库公司相继成立，
包括人大金仓、武汉达梦等